Big Data Final Project[1]

[1] Please pick one project

Evaluate the reproducibility of deep learning (100 points)

Deep learning faces significant reproducibility challenges due to multiple sources of randomness. While techniques such as seeding can provide some control, they only address a fraction of the issue. Randomness in deep learning arises from various factors, including loss function optimization, the training process, and hardware accelerations such as GPU speedup. Despite these challenges, identifying which deep learning models exhibit higher reproducibility is both desirable and critical. Such insights could guide the development and selection of models better suited for applications requiring consistent and reliable results.

Correct learning reproducibility(CLR) is the ratio that a model correctly does classification after n runs. Here we can define each run has 100 epochs. CLR can be represented as the average classification ratios of n runs.

- [2]Suppose a deep learning model has n runs and its accuracies across n runs are α_1,$\alpha_2 \cdots \alpha_n$, then CLR of this DNN is $\frac{\sum_{i=1}^{n} \alpha_i}{n}$

Datasets

We have following datasets

- MNIST data
- CIFAR-10 data
- CIFAR100 data
- KOLOD is a publicly available dataset of single-cell RNA sequencing (scRNA-seq) data. It has 704 samples across 3 classes: embryonic stem cells, neural progenitor cells, and neural crest cells. (0,1,2)

[2]There exists other ways to model this CLR also

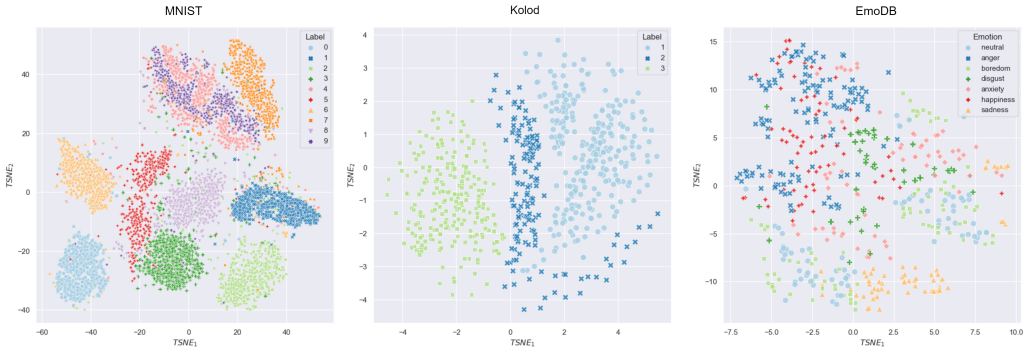


Fig 1. t-SNE visualization of three datasets

Complete the following assignments

- Calculate their learning-hard index (LHI)
- Use DNN, CNN, LSTM, GRU, LNN, Transformer models. Design and build your model architecture according to the data and present your reason. It is recommended to use relu functions. Draw each model's architecture.
  - You are strongly encouraged to include convolutional and pooling layers in the LSTM and GRU for MINIST, CIFAR10, and CIFAR-100 data, because they are image data that needs more effective feature extraction.
- You can determine to use which optimizers and loss functions but SGD should be included, but you must do seeding in your coding.
  - Also recommend to set:
  - import os
  - os.environ['TF_DETERMINISTIC_OPS'] = '1
- Run each model across the datasets for 10 runs (decrease this to 5 runs for CIFAR100 data), where each run has 100 epochs at least
  - Calculate their CLRs for 10 runs; Calculate d-index for each run for each model and visualize them
  - Which models are more reproducible, why?
- If you keep the results of the first run (the predicted labels after 100 epochs) as the reference labels—i.e., treat these as the 'correct labels,' as you did in your correct learning reproducibility—and then calculate the learning reproducibility, what can you find? Why? Note: Such learning reproducibility is intended to validate whether a deep learning model can reproduce well rather than reproduce correctly.

-
- Calculate the Weinstein, Euclidean distance, and cosine distance between the training loss and test loss curves of the first run for DNN, LSTM, GRU, and CNN; what can you find?
  - The Wasserstein distance, also known as the Earth Mover's Distance (EMD), measures the minimum "effort" required to transform one probability distribution into another. It is useful for comparing data distributions and capturing data geometry.
  - Hint: from scipy.stats import wasserstein_distance (calculate 1-Wasserstein distance)

Wasserstein Distance

The Wasserstein distance of order p between two probability distributions P and Q on a metric space $\mathcal{X}$ is defined as:

$$W_p(P,Q) = \left(\inf_{\gamma \in \Pi(P,Q)} \int_{\mathcal{X} \times \mathcal{X}} d(x,y)^p \, d\gamma(x,y) \right)^{1/p},$$

where:

- $d(x,y)$: Distance metric (e.g., Euclidean).
- $\Pi(P,Q)$: Set of all couplings γ with marginals P and Q.
- p: Order of the distance (e.g., $p = 1$ or $p = 2$).

For $p = 1$ (1-Wasserstein distance), it simplifies to:

$$W_1(P,Q) = \inf_{\gamma \in \Pi(P,Q)} \int_{\mathcal{X} \times \mathcal{X}} d(x,y) \, d\gamma(x,y).$$

This measures the minimum "work" needed to transform P into Q.

Predict AI stocks with deep learning (100 points)

- We have following three stock datasets: TAN, GOOG, SOXL

1. Partition the original data as 80% training and 20% test data as before. Use DNN, CNN, LNN, GRU, Transformer, and LSTM, Extra tree and XGB to predict their stock prices
   (a) You need to give solid reason for your deep learning network architecture (why do you choose this rather than that?)
   (b) Compare their learning behaviors and what can you find
   (c) Compare their R^2,MSEs for the test data and draw your conclusion
2. **Tune** each model's parameters so that you can enhance your model performance at least 3% and compare their learning behaviors.
3. Add meaningful features to your training and test data and tune your model again to get better performance (How to add extra features (feature interpolations), see `https://www.sciencedirect.com/science/article/abs/pii/S0020025524012003` or its free access version: `https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4256777`
4. Build your own energy efficiency index (EEI) to evaluate each model's energy cost and use real data and visualization to support your conclusion. You need to address the rationality of your EEI. Can you extend your EEI to other deep learning models?

What should you turn in?

- 1. A folder that contains
  - A ppt to show details of your analytics (at LEAST 50 pages)
  - All your data
  - workable source files (with.ipynb format); Non-workable codes will lead to zero credits
  - corresponding related output.
  - A link to your presentation video
- 2. Send the zipped file (.zip instead of ,rar)